Technical report — NLI over Swiss federal voting booklets with Apertus

What we built, how it works, and what the numbers say. Every number comes from a recorded run in `docs/runs/` (short names in brackets, listed under References) on the dev split, the validation set (val), the probe set or all three; **the sealed test dates have not been run**.

- **Track:** `Track 2A — OST: Multilingual Natural Language Inference over Swiss Official Voting Booklets`
- **Event:** Online
- **Project:** TODO project name
- **Team:** TODO team name — Rashad, Kaan (TODO full names)
- **Demo:** none

1. Summary

Given a claim in German, French or Italian and a source from an official Swiss federal voting booklet, the system decides whether the source supports (entailment, 0), does not settle (neutral, 1) or refutes (contradiction, 2) the claim. The source is a passage (task B) or a whole booklet PDF (task A). **Apertus v1.5 makes every decision.** Everything else is local and runs on CPU in the Docker image: reading the PDF, choosing the part of the booklet Apertus reads, and turning the paragraphs Apertus cites into evidence quoted verbatim with their 1-based PDF page. Apertus also names the part a claim refers to when the rules do not know the claim's opening, reads a booklet's contents when our parser cannot split it, and translates a claim for the keyword search of the fallback (section 4).

The central experiment, full booklet against selected context, decided the design: on 100 dev cases [E1-curve] the whole booklet scored 0.858 with 37,586 input tokens per case, the part the claim names (`section-route`) 0.989 with 1,248; without booklet text Apertus scores 0.435 [E2-cb].

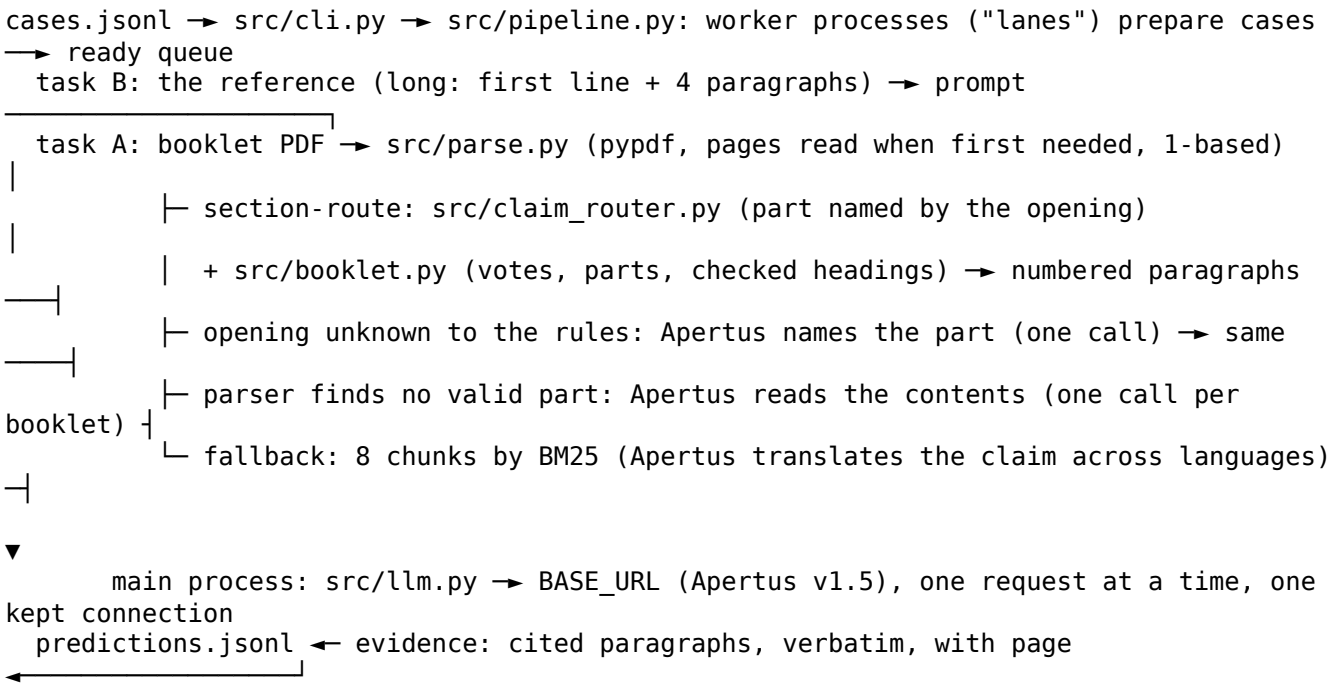
Final defaults, published image [FIN]	dev	val	probe (2 unseen dates)
Task A Macro-F1 (weight 0.30 of the score)	**0.977**	**0.961**	**0.971**
Task A evidence (0.12)	**0.980**	**0.960**	**0.969**
Task A tokens per case, input + output (K, 0.09)	1,240 + 14.7	1,211 + 14.7	1,169 + 16.5
Task A local time per case, model at 1.0 / 0.3 s per request (T, 0.09)	0.017 / 0.160 s (repeat: 0.113)	0.009 / 0.010 s	0.040 / 0.028 s
Task B Macro-F1 (0.28)	**0.987**	**0.969**	**0.977**
Task B tokens per case, input + output (K, 0.06)	815 + 8.7	820 + 9.4	789 + 8.6
Task B local time per case, 1.0 / 0.3 s per request (T, 0.06)	0.012 / 0.010 s	0.007 / 0.007 s	0.015 / 0.016 s
Labels written by Python (neutral fallback)	0 of 600	1 of 1,160	0 of 188
REF SCORE at 1.0 / 0.3 s per request (section 2)	**0.903 / 0.845**	**0.913 / 0.907**	**0.897 / 0.896**

Sealed test dates (2020-02-09, 2022-02-13, 2022-05-15): TODO, results after the organisers' evaluation.

2. What is scored, and what we optimise

The organisers' score (Q&A of 8 October; `docs/official_scoring.md`): **Score = 0.4 S_B + 0.6 S_A**, with S_A = 0.50 Macro-F1 + 0.20 evidence + 0.15 T + 0.15 K and S_B = 0.70 Macro-F1 + 0.15 T + 0.15 K. K = the best team's tokens per case ÷ ours; T = the best local time per case ÷ ours, where **local time is the container's wall-clock time minus the time during which a model request is in flight**: start-up, PDF reading, embedding and every gap between two requests count. Task A and task B run in separate containers. To rank our own versions we use **REF SCORE** (`scripts/official_score.py`): the same formula with "best" values fixed by us before any measurement (600 / 800 tokens, 0.05 / 0.005 s local time per case for task A / B). Local time is measured the organisers' way (`scripts/local_time.py`): the image with 2 CPUs and 4 GB against a fake model that replays saved answers after 1.0 s (about Public AI's speed), 0.3 s or 0.1 s per request.

3. Architecture



- `src/cli.py`: the entrypoint; each case is `prepare()` (local work) then `complete()` (model calls); every id is answered (neutral if all else fails), each answer written at once, exit code 0.
- `src/pipeline.py`: two forked lanes, each owning whole booklets, prepare cases while the main process sends them; one lane under 2.5 GiB. A **memory guard** (`src/rankers.py`) does without an embedding model whose need (e5: 1.5 GiB) exceeds the container's limit or that fails to load: long parts send their first 8 paragraphs, long task B references go whole. Caches are keyed by content; replaying the saved answers of all 1,180 dev and val requests reproduces every request, label and evidence (G1, G2).
- `src/llm.py`: the only module that calls a model (`BASE_URL`, temperature 0, one kept connection, retries).

3.1 How document context is prepared and supplied to the model

Parsing. pypdf reads a page's text when first needed, with its 1-based PDF page number; an unreadable page is repaired or left empty, never failing the booklet.

Selection (`section-route`, task A). (1) *Router:* the claims open with a formula naming their source ("Laut der Zusammenfassung …", "Le Conseil fédéral estime …"); regular expressions map it to one of five parts: summary, Federal Council, committee, text put to the vote, detailed explanation. If they find none, Apertus names the part (one call). (2) *Parser:* the table of contents gives each vote's sections (two layouts: before and after mid-2018); every start page is checked against its printed heading; the arguments are split by voice. If no part of the vote passes, Apertus

reads the contents pages (one call per booklet) and its start pages go through the same checks. (3) *Vote match:* the case's `vote` against the contents titles (fuzzy); if that fails, by meaning with multilingual-e5-small. (4) *Paragraphs:* the part's lines grouped into paragraphs, each with its page; a part over 8,000 characters keeps its 8 paragraphs most similar to the claim (e5). (5) *Fallback:* no part, no vote, or any error → the booklet's 8 chunks of up to 1,000 characters with the best BM25 score for the claim's words; a claim in another language than the booklet is first translated into the booklet's language by Apertus (one call), and the reading call gets the claim as it is.

What the prompt contains. System: the instruction and the decision rule (`A-v4-section-route-L1`):

> *Decide in this order: 1. Does the reference text deal with the subject of the claim at all? If not, the label is 1 (neutral). 2. If it does: 0 if it supports the claim; 2 only if it states something that cannot be true together with the claim; otherwise 1. Missing information is never a contradiction. […]*

User: `PART:` (which part, whose voice), `REFERENCE TEXT:` with paragraphs `[1] … [n]`, `VOTE:`, `CLAIM:` as given. `response_format: json_schema` forces `{"paragraphs": [≤ 3 numbers], "label": 0|1|2}`. Task B sends the same rule (`v3-topic-first`) for `{"label": n}`; a reference over 3,000 characters is sent as its first line and the 4 paragraphs whose first 300 characters are most similar to the claim (e5).

Evidence. The paragraphs Apertus cited, verbatim with their page, then their halves, up to five items; none for neutral answers and task B.

3.2 Who does what

Step	Python (rules, deterministic)	Local model (multilingual-e5-small, CPU)	Apertus v1.5
Read input, answer every id; PDF text per page	✓ (pypdf)		
Which part the claim names	✓ (regular expressions)		only when the rules find none
Which vote; where its parts start	✓ (contents, checked headings)	vote by meaning if the title match fails	reads the contents when the parser gives no valid part
Which paragraphs	✓ (the part; the fallback's chunks by BM25)	parts over 8,000 characters; task B references over 3,000 characters	translates the claim for the fallback across languages
Label	only the neutral fallback for an unreadable answer: 1 of 2,128 final labels		✓ **every label**
Which paragraphs justify it; evidence	copies what Apertus cited, with its page		✓ cites them

4. Use of Apertus

- **Model:** Apertus v1.5, named by `MODEL` (default `swiss-ai/Apertus-v1.5-8B`). Our runs used `swiss-ai/apertus-v1.5-8b` on Public AI. Which model and server the evaluation uses is not known to us.
- **How:** inference only, zero-shot, temperature 0; one call per case, plus one when the rules do not know a claim's opening, one per booklet whose vote the parser cannot split, and one to translate a claim that falls back across languages; answer budget 128 tokens (task A), 32 (task B). No fine-tuning. Reached only through `BASE_URL`.
- **Apertus as parser** (sessions 11 and 12). We tried Apertus in the roles Python plays before the answer; every parsing call counts in the case's tokens, a booklet's call in its first case:

Role	Accuracy	Effect (interleaved runs)	Tokens per case, input + output	In the final system?
AP-part: name the part a claim's opening refers to (one call per claim)	79–85 % of reworded openings (council, summary, committee 95–100 %; law about 50 %; detail 16–24 %)	claims with openings the rules do not know (150 reworded per set): Macro-F1 0.967 / 0.926 (dev / val) against 0.894 / 0.876 for the best fallback, evidence 0.880 / 0.840 against 0.670 / 0.650	+236 + 49 on such cases	**yes, after the rules** (≥ +0.02 on dev and val); no dev, val or probe claim needs it (the rules name a part for all)
AP-structure: read the contents (one call per booklet)	start pages as the parser's for 78.7 % of votes	parser off: dev 0.946 / 0.925, val 0.939 / 0.917 against 0.896 / 0.672, 0.872 / 0.593	1,825 + 192 per booklet	**yes, after the parser** (no dev, val or probe case needs it)
AP-translate: a claim into the booklet's language, for the BM25 fallback	—	fallback forced: dev 0.956 / 0.736, val 0.957 / 0.755 against 0.900 / 0.672, 0.875 / 0.588	+128 + 42 when asked	**yes**

- Not used: both roles for every case (dev 0.960 / 0.891 against 0.977 / 0.980) and reading page images.
- **Development tools:** Claude Code wrote code and ran experiments; it is never called by the pipeline.

5. Data

- **Dataset:** `OSTswiss/MNLIoverSwissVotingBooklets` v1.1 (commit `9ff08597`, MIT): 1,488 rows, 335 exact duplicates; each row gives one task A and one task B case.
- **Splits** (by voting date, duplicates removed): test 5 dates (267 rows); dev 300 rows; val 580 rows of the dev dates (never used to change a rule); **probe:** the 94 test rows of 2021-03-07 and 2025-09-28; three dates sealed.
- **Booklets:** the Federal Chancellery's PDFs, downloaded by script and not committed. For parser coverage, every federal booklet from 2016 to 2026 in de, fr and it (93 PDFs; the sealed dates never downloaded). Swiss copyright law exempts official reports of authorities (Art. 5 URG). No personal data.

6. Evaluation

Method. The starter's `evaluate.py`, unchanged. The endpoint drifts, so **only arms of one interleaved run compare** (every case answered by all versions back to back; all session 11 and 12 answers came from one backend). Rules were fixed before the runs; a version that fails stays in the code, switched off.

Parser coverage [COV] (93 booklets 2016–2026; the three languages of a date must agree): booklets whose votes all pass 64 → **88**, votes passing every check 178 → **244 of 249**, probe cases routed 83 → **90 of 94** (session 11).

Fallbacks, forced [FB-dev, FB-val, S12-dev, S12-val]: every case put into a fallback situation by a test-only switch ("no part": the router returns nothing, AP-part off; "no vote": the vote match fails; "no parser": none of the vote's parts passes its checks). Macro-F1 / evidence, dev; val (balanced 300); each line against the arm of its own interleaved run:

Situation, version	dev	val	Tokens (dev)	Adopted?
routed (today, not forced)	0.977 / 0.980	—	1,238 + 14.5	
no part: e5's 8 chunks of the booklet (before session 12)	0.900 / 0.672	0.875 / 0.588	1,868 + 15.8	replaced
no part: BM25 chunks, Apertus translates across languages	**0.956 / 0.736**	**0.957 / 0.755**	2,591 + 44.3	**yes**
no parser: e5's chunks (before session 12)	0.896 / 0.672	0.872 / 0.593	1,868 + 15.8	replaced
no parser: Apertus reads the contents	**0.946 / 0.925**	**0.939 / 0.917**	1,564 + 42.8	**yes**

Not adopted: vote-wide (0.927 / 0.873) and booklet-wide paragraphs (0.857 / 0.856); model S's chunks (0.913, dev).

Task B, fewer tokens [TB-dev, TB-val]: a reference over 3,000 characters is cut to its first line and the 4 paragraphs whose first 300 characters are most similar to the claim (T): Macro-F1 dev 0.987, val 0.968 with 815 input tokens per case, against 0.967, 0.954 with 1,994 for the whole reference.

Local time (the organisers' measure; image, 2 CPUs, fake model replaying the final answers) [FIN, STEP4, S-COST]. What is local in task A is PDF reading (about 1.3 s per booklet, pure-Python pypdf) and e5's embedding of long parts (about 3 s each); session 11 hid them better behind the requests (0.202 → 0.101 s per case at 0.3 s per request, same requests), and session 12's BM25 fallback removed e5's whole-booklet embedding (probe: 0.237 → 0.028 s).

Final image, local time per case	1.0 s per request	0.3 s	0.1 s
Task A, dev	0.017 s	0.160 s (repeat 0.113 s)	0.364 s
Task B, dev	0.012 s	0.010 s	0.161 s
Task A with model S instead of e5 (not adopted, step 4)	0.016 s	0.018 s	0.043 s

Single measurements at 0.3 s vary by up to about ±0.03 s.

Memory [MEM, GUARD]: each lane peaks at about 1.2 GiB, almost all of it e5; one lane under 2.5 GiB; below 1.5 GiB (or without e5's files) no e5: long parts send their first 8 paragraphs, long task B references go whole. Through the image, both tasks write every response at 4g, 2g, 1.5g, 1g, 768m and 512m, with 2 CPUs and with 1 (session 11's image lost 24 task A and 190 task B responses at 1 GiB).

Versions tried in sessions 11 and 12 (each by a rule fixed before its run; a version that fails stays in the code, off):

Version	Rule, outcome
Parser fixes; light-first, lazy pages, shared CPUs; one lane under 2.5 GiB; memory guard	same answers where they apply, the measure improves: **adopted**
AP-part after the rules; task B selector T; BM25 fallback; AP-structure after the parser	their rules on dev and val: **adopted**

Version	Rule, outcome
Vote-/booklet-wide fallback; AP-structure for every case; quantised e5; task B E, Q, H (+0.007, +13 % tokens), B-R, B-BM25	fail their rules: off
Model S for task A: dev 0.973 / 0.970 against 0.980 / 0.980; local time at 0.3 s 0.018 against 0.144 s; memory 200 against 2,286 MiB	fails by one case (−0.0067; allowed −0.005): off (`--ranker S`)

Probe: two unseen voting dates (test rows of 2021-03-07 and 2025-09-28, opened in session 10). Through the final image [FIN-probe]: task A **0.971**, evidence **0.969** (63 of 65), task B **0.977**; REF SCORE **0.897 / 0.896**. Session 11's image: 0.971, 0.954, 0.977. 90 of 94 task A cases are routed; the 4 others are claims about a committee in votes that have none, answered by the BM25 fallback (all 4 right).

7. Limitations

- **Tuned on dev.** Prompts, router patterns and parser checks were written with dev in view; val confirms; the probe (two unseen dates) lands at val's level.
- **`section-route` depends on the booklet layout** (2016–2026 checked: 244 of 249 votes pass). A claim whose opening neither the rules nor Apertus place, or a vote the parser cannot read, falls back (answered, less accurate).
- **Endpoint and speed.** Our scores hold for the backend that answered; with a fast model, PDF reading and e5's embedding no longer hide behind the requests. Model S would remove the latter (0.018 against 0.144 s per case at 0.3 s per request) but lost one dev case of 300 to e5, below its rule.
- **Not political advice.** The system checks a claim against the booklet's text only. Its answers are not voting recommendations. Every task A label 0 or 2 comes with the booklet page and the verbatim text it rests on.

8. Reproducibility

- **Published image:** `ghcr.io/rashad18236/hackapertus-voting-nli:latest` (built from `main` and tested on a fresh runner by `.github/workflows/publish-image.yml`), run with the organisers' command (README); this report's numbers come from `sha-828429cab766` (session 12 branch).
- **From source:** `make run` builds the image and answers `examples/cases.jsonl`.
- **Hardware:** CPU only; 2 CPUs and 4 GB measured (peak 2.3 GiB with two worker processes); the image answers every case down to 512 MiB, with 2 CPUs or 1: below 1.5 GiB without the embedding model [MEM, GUARD].
- **Determinism:** temperature 0; a fake model replays saved answers by request hash (G1, G2); 317 tests.

License

Code: Apache-2.0 (the repository's `LICENSE`). Report text: Creative Commons Attribution 4.0 (CC-BY-4.0).

References

- Dataset: OSTswiss/MNLIoverSwissVotingBooklets v1.1, Hugging Face (MIT). Starter: `gitlab.com/ifsoftware/hackapertus-starter`. Apertus v1.5, Swiss AI Initiative.
- Wang, L. et al. (2024). *Multilingual E5 Text Embeddings.* arXiv:2402.05672 (MIT). Model S: `static-similarity-mrl-multilingual-v1` (Apache-2.0). Robertson, S., Zaragoza, H. (2009). *BM25 and Beyond.*
- Runs (`docs/runs/`, dated 2026-10-09 to 13): FIN `final-s12_*`; S12 `S-and-fallbacks_devA300`, `bm25ap-fallback_valA300`, `ap-structure-ask_*`; GUARD `memory-guard_grid48`; S-COST `S-cost_devAB600`; STEP4, MEM `local-time-step4_devA300`, `memory-step5_devA300`; TB `taskb-selectors_*`; COV `parser-coverage_booklets93`; FB `fallbacks_*`; E1-curve `context-curve_devA100`; E2-cb `closed-book_devA300`.